



Drools na prática: Criando um validador de CNH com Java

Recentemente, resolvi explorar um pouco mais o **Drools** e entender, na prática, como funciona um motor de regras dentro de uma aplicação Java.

Eu já conhecia a ideia de separar regras de negócio do código da aplicação, mas queria enxergar melhor conceitos como **Facts**, **Working Memory**, **Agenda**, **KieSession** e **modify()** funcionando juntos.

Para isso, criei um projeto simples: uma aplicação que recebe uma lista de **Carteiras Nacionais de Habilitação (CNHs)** e utiliza regras para determinar se cada uma delas deve ser considerada válida ou inválida.

O projeto foi desenvolvido utilizando **Java 8**, **Maven** e **Drools 7.73.0.Final**.

Vale destacar que este projeto não tenta reproduzir as regras oficiais de validação de uma CNH. Meu objetivo foi criar um cenário simples para estudar o funcionamento de uma Rule Engine.

Por que decidi usar o Drools?

Quando penso em regras de negócio dentro de uma aplicação, uma das primeiras soluções que vêm à mente são condicionais.

Eu poderia, por exemplo, escrever algo assim:

```
if (cnh.isExpirada()) {  
    cnh.setStatus(Status.INVALIDA);  
}  
  
if (cnh.getPontos() > 20) {  
    cnh.setStatus(Status.INVALIDA);  
}
```

Para um cenário pequeno, não existe nada necessariamente errado nisso.

O problema começa quando as regras aumentam.

Imagine dezenas ou até centenas de condições, com diferentes combinações, prioridades e dependências. Aos poucos, a lógica de negócio começa a ficar espalhada pelo código da aplicação.

Foi justamente essa separação que eu quis experimentar com o Drools.



Em vez de deixar o Java responsável por todas as decisões, minha ideia foi chegar a algo conceitualmente parecido com:

Dados
+
Regras de negócio
+
Motor de regras

Assim, os objetos representam os dados, as regras representam as decisões e o Drools fica responsável por avaliar quais regras devem ser executadas.

O cenário

Para manter o projeto simples, defini apenas algumas informações para uma CNH:

- ✓ numero
- ✓ nome
- ✓ pontos
- ✓ dataExpiracao
- ✓ status
- ✓ causa

Também criei o método:

isExpirada()

que permite verificar se a data de validade da CNH já passou.

Toda CNH entra no processamento inicialmente com o status:

NAO_PROCESSADA

Depois que o Drools executa as regras, esse estado pode mudar para:

VALIDA

OU:

INVALIDA

Com essa estrutura pronta, comecei a criar as regras.



Minha primeira regra: CNH expirada

A primeira situação que quis validar foi bastante simples: verificar se a CNH estava expirada.

No arquivo `rules.drl`, posso utilizar diretamente o método que criei no objeto:

`Cnh(isExpirada())`

Então montei uma regra conceitualmente assim:

```
rule "Cnh Expirada"
when
    $cnh: Cnh(isExpirada())
then
    modify($cnh) {
        setStatus(Status.INVALIDA),
        setCausa("CNH expirada")
    }
end
```

Quando uma CNH satisfaz essa condição, altero seu status para `INVALIDA` e registro a causa.

Foi aqui que comecei a perceber uma diferença importante entre simplesmente executar condicionais no Java e trabalhar com uma Rule Engine.

Eu não estou dizendo explicitamente:

```
Execute a regra 1.
Depois execute a regra 2.
Depois execute a regra 3.
```

Eu descrevo **condições**.

O Drools fica responsável por descobrir quais regras são aplicáveis aos objetos disponíveis.

Depois adicionei a regra de pontos

Minha segunda regra verifica se a CNH possui mais de 20 pontos.

A condição ficou bastante simples:

`Cnh(pontos > 20)`

E a regra segue a mesma ideia:

```
rule "Cnh pontos"
when
    $cnh: Cnh(pontos > 20)
then
```



```
modify($cnh) {  
    setStatus(Status.INVALIDA),  
    setCausa("Cnh tem mais pontos que o permitido de 20")  
}
```

end

Se a condição for verdadeira, a CNH também será marcada como inválida.

Nesse momento, comecei a enxergar melhor uma das vantagens dessa abordagem.

Minha classe Cnh não precisa saber que possuir mais de 20 pontos significa determinada coisa dentro do negócio.

Ela fornece seus dados.

A decisão está na regra.

Isso cria uma separação interessante entre **modelo de dados e decisão de negócio**.

E como determinei que uma CNH é válida?

Aqui apareceu uma situação um pouco mais interessante.

Toda CNH começa como:

NAO_PROCESSADA

Se uma regra impeditiva encontrar algum problema, esse status será alterado.

Portanto, no exemplo, se o objeto continuar como *NAO_PROCESSADA*, posso considerá-lo válido.

Criei então uma terceira regra:

```
rule "Cnh valida"  
when  
    $cnh: Cnh(status == Status.NAO_PROCESSADA)  
then  
    modify($cnh) {  
        setStatus(Status.VALIDA),  
        setCausa("Cnh sem status, considerada valida por enquanto")  
    }  
end
```

Essa regra também me fez perceber algo importante.

Regras podem depender do estado que outras regras modificam.



E isso muda bastante a maneira como precisamos pensar sobre o fluxo da aplicação.

Colocando os objetos dentro do Drools

Com as regras prontas, precisei criar a sessão que executaria tudo.

No kmodule.xml, configurei uma base e uma sessão:

```
<kbase packages="org.example">
  <ksession name="session"/>
</kbase>
```

Depois, na aplicação, criei a KieSession:

```
kieContainer.newKieSession("session");
```

A partir daí, comecei a inserir as CNHs:

```
for (Cnh cnh : listaCnh) {
    kieSession.insert(cnh);
}
```

E finalmente:

```
kieSession.fireAllRules();
```

Essa pequena linha:

```
fireAllRules();
```

é onde boa parte da mágica começa a acontecer.

Entendendo a Working Memory

Um conceito que ficou muito mais claro para mim depois desse projeto foi a **Working Memory**.

Quando faço:

```
kieSession.insert(cnh);
```

estou disponibilizando aquele objeto para o motor de regras.

No meu exemplo, tenho quatro CNHs.



Posso imaginar a memória de trabalho desta maneira:

Working Memory

```
— CNH 1  
— CNH 2  
— CNH 3  
— CNH 4
```

Esses objetos passam a ser os **Facts** que minhas regras podem analisar.

Uma regra procura:

```
Cnh(isExpirada())
```

Outra procura:

```
Cnh(pontos > 20)
```

E outra:

```
Cnh(status == Status.NAO_PROCESSADA)
```

O Drools confronta os fatos disponíveis com as condições declaradas nas regras.

Foi nesse momento que a ideia de um motor de inferência começou a ficar mais concreta para mim.

E onde entra a Agenda?

Outro conceito que eu queria entender melhor era a **Agenda**.

Antes desse projeto, era fácil imaginar o **.drl** quase como um arquivo sendo executado linha por linha.

Mas essa não é uma boa maneira de pensar no Drools.

Quando chamo:

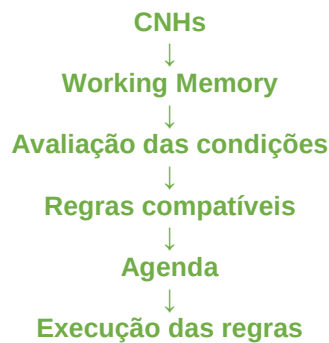
```
fireAllRules();
```

o Drools avalia os fatos disponíveis e identifica quais regras possuem condições satisfeitas.

Essas correspondências podem gerar **ativações**, que são administradas pela Agenda.



Simplificando bastante, passei a visualizar o processo assim:



Essa mudança de perspectiva foi importante.

Em vez de pensar:

“Qual regra vem primeiro no arquivo?”

passei a pensar:

“Quais regras podem ser ativadas considerando o estado atual dos meus fatos?”

Essa segunda pergunta faz muito mais sentido quando estamos trabalhando com um motor de regras.

O `modify()` foi outro conceito importante

Durante o desenvolvimento, um dos pontos que mais me chamou atenção foi o `modify()`.

Por exemplo:

```
modify($cnh) {  
    setStatus(Status.INVALIDA),  
    setCausa("CNH expirada")  
}
```

Inicialmente, podemos olhar para isso apenas como uma maneira diferente de alterar propriedades do objeto.

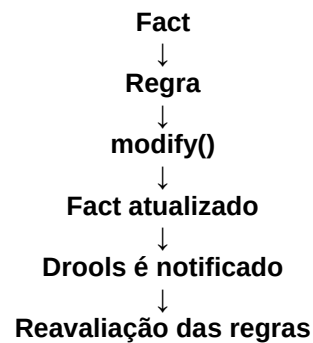
Mas existe algo mais importante acontecendo.

Quando utilizo `modify()`, além de alterar o objeto, estou notificando o Drools de que aquele Fact foi atualizado.

Isso significa que regras que dependem das propriedades alteradas podem ser reavaliadas.



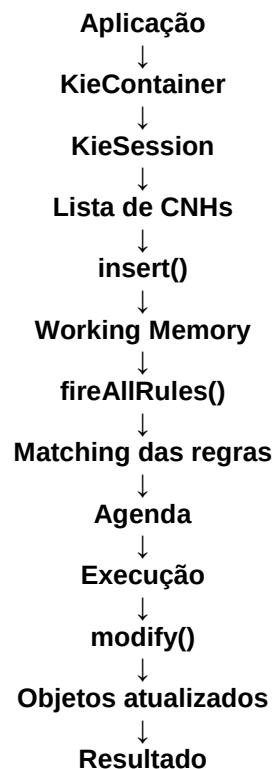
Passei a visualizar isso assim:



E isso explica por que precisamos ter cuidado quando várias regras modificam o mesmo objeto.

O fluxo completo que consegui visualizar

Depois de juntar todas essas peças, cheguei a uma visão muito mais clara do fluxo da aplicação:



Esse fluxo também aparece representado no diagrama da página 3 do documento que produzi para o projeto.

Foi provavelmente a parte que mais me ajudou a conectar os conceitos.

```

1 package org.example
2
3 import org.example.Cnh;
4 import org.example.Status;
5 import java.time.LocalDate;
6
7 rule "Cnh expira"
8 dialect "java"
9 when
10     $cnh: Cnh(isValidpra())
11 then
12     //System.out.println("$cnh + " INVALIDA porque expiro!");
13     modify($cnh) {
14         setStatus(Status.INVALIDA);
15         setCause("Cnh expira");
16     }
17 end
18
19 rule "Cnh pontos"
20 dialect "java"
21 when
22     $cnh: Cnh(pontos > 30)
23 then
24     //System.out.println("$cnh + " INVALIDA por ter mais pontos que o permitido
25     modify($cnh) {
26         setStatus(Status.INVALIDA);
27         setCause("Cnh tem mais pontos que o permitido de 30");
28     }
29 end
30
31 rule "Cnh valida"
32 dialect "java"
33 when
34     $cnh: Cnh(status == Status.NAO_PROCESSADA)
35 then
36     //System.out.println("$cnh + " sem validar, considerar VALIDA por enquanto
37     modify($cnh) {
38         setStatus(Status.VALIDA);
39         setCause("Cnh sem status, considerada valida por enquanto");
40     }
41 end
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
98
```



Uma coisa que eu mudaria em uma próxima versão

Durante esse estudo, também encontrei um ponto que merece atenção.

Eu não defini salience nas regras.

Ou seja, não existe uma prioridade explícita configurada entre elas.

Isso é particularmente interessante porque minha regra de CNH válida depende desta condição:

```
status == Status.NAO_PROCESSADA
```

Enquanto outras regras podem modificar justamente esse status.

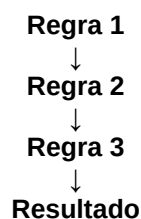
Portanto, eu não deveria construir uma solução maior dependendo apenas da ideia de que determinada regra “está antes” ou “está depois” dentro do arquivo .drl.

Em uma próxima evolução do projeto, eu exploraria melhor recursos como salience, agenda-group e activation-group, além de criar testes específicos para cenários nos quais várias regras possam ser verdadeiras ao mesmo tempo.

O principal aprendizado para mim não foi simplesmente descobrir como escrever uma regra .drl.

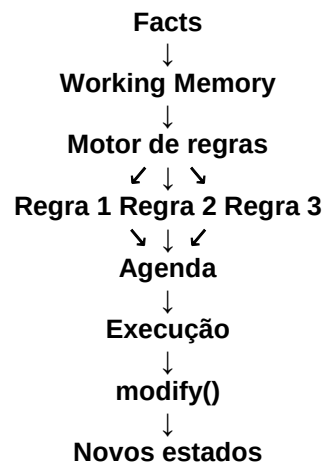
Foi entender melhor **como pensar em uma aplicação orientada a regras**.

Antes, eu poderia imaginar algo assim:





Depois desse experimento, passei a enxergar algo mais próximo de:



Essa diferença parece pequena, mas muda bastante a forma como projetamos a solução.

Ainda há bastante coisa que pretendo explorar a partir desse projeto.

Quero adicionar mais regras, criar testes automatizados para cada cenário e, principalmente, experimentar situações em que **duas ou mais regras sejam ativadas para o mesmo objeto**.

Também quero explorar melhor mecanismos de organização e controle da execução das regras.

Uma evolução interessante seria transformar o exemplo em uma API, permitindo enviar uma CNH por uma requisição e deixar o Drools realizar a avaliação.

Esse projeto foi criado para que você possa entender melhor como o Drools funciona na prática.

Mais do que substituir alguns `if/else`, comecei a enxergar o Drools como uma maneira diferente de organizar determinadas decisões de negócio.

No exemplo que construí, consegui separar claramente as responsabilidades:

Java

→ controla o fluxo da aplicação

Objetos

→ representam os dados

DRL

→ concentra as regras de negócio

Drools

→ avalia os fatos e executa as regras aplicáveis



E conceitos que antes pareciam isolados — **Facts, Working Memory, Agenda, KieSession, fireAllRules() e modify()** — passaram a fazer parte de um único fluxo.

O projeto ainda é simples, mas cumpriu exatamente o que eu queria: **tirar o Drools da teoria e enxergar, passo a passo, o que acontece dentro de uma Rule Engine.**

EducaCiência FastCode para a comunidade